AI Assignment 01 : 3SAT–Problem

資工二乙 張惟喆

學號：412262371

1. 實作結果截圖：

DFS 實作截圖

```
-------- result of DFS --------
Dimension: 10
Solution Found!
1 0 0 1 0 0 1 0 1 0
we have expanded 15/1000 nodes
Time usage: 292 ms
Space cost(max_depth): 10
------------------------------
Dimension: 20
Solution Found!
1 0 0 1 0 0 0 1 1 0 0 0 0 1 0 1 0 0 0 1
we have expanded 46/8000 nodes
Time usage: 1800 ms
Space cost(max_depth): 20
------------------------------
Dimension: 30
Solution Found!
0 0 0 0 1 0 0 1 0 0 0 1 1 0 0 0 0 1 0 1 0 1 1 0 1 0 1 0 1 0
we have expanded 58/27000 nodes
Time usage: 2856 ms
Space cost(max_depth): 30
------------------------------
Dimension: 40
Solution Found!
0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 1 1 0 0 0 1 1 0 0 0 0 1 0 1 0 0 0 0 1 1 0 1
we have expanded 5942/64000 nodes
Time usage: 320565 ms
Space cost(max_depth): 40
------------------------------
Dimension: 50
Solution Found!
1 1 1 1 1 1 1 0 0 1 0 1 1 1 0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 1 0 1 0 1
we have expanded 47302/125000 nodes
Time usage: 2863125 ms
Space cost(max_depth): 50
------------------------------
```

BFS 實作截圖：

```
-------- result of BFS --------
Dimension: 10
Solution Found!
1 0 0 1 0 0 1 0 1 0
we have expanded 15/1000 nodes
Time usage: 309 ms
Space Cost(max priority queue size): 5
------------------------------
Dimension: 20
Solution Found!
1 0 0 1 0 0 0 1 1 0 0 0 0 1 0 1 0 0 0 1
we have expanded 46/8000 nodes
Time usage: 1773 ms
Space Cost(max priority queue size): 11
------------------------------
Dimension: 30
Solution Found!
0 0 0 0 1 0 0 1 0 0 0 1 1 0 0 0 0 1 0 1 0 1 1 0 1 0 1 0 1 0
we have expanded 58/27000 nodes
Time usage: 2891 ms
Space Cost(max priority queue size): 16
------------------------------
Dimension: 40
Solution Found!
0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 1 1 0 0 0 1 1 0 0 0 0 1 0 1 0 0 0 1 1 0 1
we have expanded 5943/64000 nodes
Time usage: 249992 ms
Space Cost(max priority queue size): 17
------------------------------
Dimension: 50
Solution Found!
1 1 1 1 1 1 1 0 0 1 0 1 1 1 0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 1 0 1 0 1
we have expanded 47302/125000 nodes
Time usage: 2404728 ms
Space Cost(max priority queue size): 22
------------------------------
```

A*實作截圖：

```
-------- result of A* --------
Dimension: 10
Solution Found!
1 0 0 1 0 0 1 0 1 0
we have expanded 22/1000 nodes
Time usage: 348 ms
Space Cost(max priority queue size): 8
------------------------------
Dimension: 20
Solution Found!
1 0 0 1 0 0 0 1 1 0 0 0 0 1 0 1 0 0 0 1
we have expanded 58/8000 nodes
Time usage: 1940 ms
Space Cost(max priority queue size): 14
------------------------------
Dimension: 30
Solution Found!
0 0 0 0 1 0 0 1 0 0 0 1 1 0 0 0 0 1 0 1 0 1 1 0 1 0 1 0 1 0
we have expanded 77/27000 nodes
Time usage: 4667 ms
Space Cost(max priority queue size): 20
------------------------------
Dimension: 40
Solution Found!
0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 1 1 0 0 0 1 1 0 0 0 0 1 0 1 0 0 0 1 1 0 1
we have expanded 5691/64000 nodes
Time usage: 238935 ms
Space Cost(max priority queue size): 58
------------------------------
Dimension: 50
Solution Found!
1 1 1 1 1 1 1 0 0 1 0 1 0 1 0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 1 0 1 0 1
we have expanded 28701/125000 nodes
Time usage: 1252346 ms
Space Cost(max priority queue size): 70
------------------------------
```

2. Cost 分析

因為所有演算法都有找到解，沒辦法比較達到上限時的解品質，所以我在程式碼中另外加了計算 Space Cost，來計算不同演算法的 Cost 有甚麼不同

Space Cost	DFS (call sack)	BFS (PQ max)	A* (PQ max)
D = 10	10	5	8
D = 20	20	11	14
D = 30	30	16	20
D = 40	40	17	58
D = 50	50	22	70

A* 所使用 g_cost + h_score 應該使得每次挑選的節點都是最正確的，理論上 Space Cost 應該要最小，但是在這些測試結果中，BFS 的 Space cost 反而比 A* 還要少。這個部分我研究了一陣子還是不知道為甚麼，不過我的猜測是 A* 裡面的 g_cost 和 h_score 的權重不同，演算法偏向保留某些節點，導致 space cost 變多。

DFS 因為每次都是一條路走到底，所以它雖然記憶體少，但在 dimension 越高的時候，這樣子的找法會非常花時間。

3. Expanded nodes 分析

Expanded nodes	DFS	BFS	A*
D = 10	15	15	22
D = 20	46	46	58
D = 30	58	58	77
D = 40	5942	5943	5691
D = 50	47302	47302	28701

經過這些測試，DFS 所展開的節點數數量和 BFS 幾乎一樣。得益於 DFS 有一個 `decide_the_value_order()` 的函式來幫助 DFS 選擇要優先嘗試哪一個值，這個影響 DFS 很多，在沒有這個函式之前，DFS 跑所有的測資負擔都相當大。雖然這麼加了這個會讓 DFS 有點像是 BFS，但考慮到他還是一路向下走到底，所以我認為這還算是 DFS 的一種。

Slow_DFS	D=10	D=20	D=30	D=40	D=50
Expanded nodes	15	902	4157	5698	125000 (沒找到解)

取消 `decide_the_value_order()` 之後 DFS 所需要展開的節點數

三種演算法都能在限制內找到解，但 A* 明顯展開的節點數最少，代表它的 heuristic 導向效果最好。

4. Running Time 分析

Running Time	DFS	BFS	A*
D = 10	0.000202 s	0.000221 s	0.000313 s
D = 20	0.000944 s	0.001081 s	0.001399 s
D = 30	0.001692 s	0.001670 s	0.002654 s
D = 40	0.252628 s	0.188026 s	0.206444 s
D = 50	2.345220 s	1.759090 s	1.175000 s

根據這些結果，可以發現在 Dimension 不大的時候，時間花費：

DFS < BFS < A*

這時候 DFS 的速度都是最快的，但基本和 BFS 持平。

隨著 Dimension 逐漸上升，時間花費：

A* < BFS < DFS

這時候反而是 A* 每次都花時間計算出更正確的路會使的總時間較短。所以當問題的範圍越來越大的時候，我們需要更好的搜尋策略，才不會像 DFS 一樣容易一直搜尋錯的分支，導致耗費大量的時間

考慮到 DFS 在每次探索新的點之前都會評估優先探索的值，也會花一些時間。所以我嘗試把 decide_the_value_order() 拿掉，再做一次時間的測試

Slow_DFS	D=10	D=20	D=30	D=40	D=50
Running Time	0.000103 s	0.006896 s	0.045417 s	0.086075 s	2.1808 s (沒找到解)

取消 decide_the_value_order() 之後 DFS 所需要花費的總時間

這時候 DFS 在深度不深的時候執行時間的優勢就很明顯，所以 DFS 適合小範圍的問題，可以快速得到解。

5. 實作歷程

- **弄不清楚 assignment_still_viable() 和 satisfied_all_clauses() 的差別:**

  最一開始的時候我的 BFS 並沒有區分這兩個函式，我覺得我只需要紀錄目前滿足所有 clauses 的節點就好，這樣可以最大程度的加快我搜尋到解的時間。而在 num_of_satisfied_clauses()裡我把還未 assigned 的變數也視為能夠滿足 clause(還有機會就算可以)。也就是整串還未指定任何變數的 assignment = 可以滿足所有 clauses。嚴重高估的 heuristic 讓我收集了一堆分數高而且沒用的點，後果可想而知。最後靠著限制 pq 的 size 才順利勉強把節點數壓縮進 D^3。

  所以 assignment_still_viable()是決定有沒有機會滿足(還沒 assigned 也算有機會)，而 num_of_satisfied_clauses()作為 heuristic function 的評估函式，必須要完全滿足(不包含-1)才不會高估分數導致節點爆炸。

- **Heuristic function 的測試和修改：**曾經嘗試用多種因素來評估當前 assignment 的潛力，例如 (滿足 clauses 數量 + clause 裡變數出現頻率) 或加入懲罰分數 (無法滿足的 clauses 數量)，最後都效果一般，甚至有反效果。我覺得可能是因為我不知道如何評估這些分數的權重，或因為錯誤計算 `num_of_satisfied_clauses()`，直接導致不論我如何努力，結果的改變都不顯著。
- **DFS 的節點數過多：**純粹暴力的 `DFS` 讓我在 `Dim = 20` 的時候就卡住了。但是經過 `BFS` 和 `A*` 的實作，我已經很知道該如何有效的剪枝。除了優先探索在 `clauses` 裡出現次數較多的變數以外，針對每個新的 `assignment`，只需要嘗試一個「最高優先且還未被決定」的變數，就可以離開迴圈了。避免在錯誤的方向還每一層都依序嘗試完所有變數。